PRD: Claude
Holding the Thread Across a Revision

Giving Claude a memory of its own edits, so iteration stays trustworthy at every round

Author : Bharat Kalra

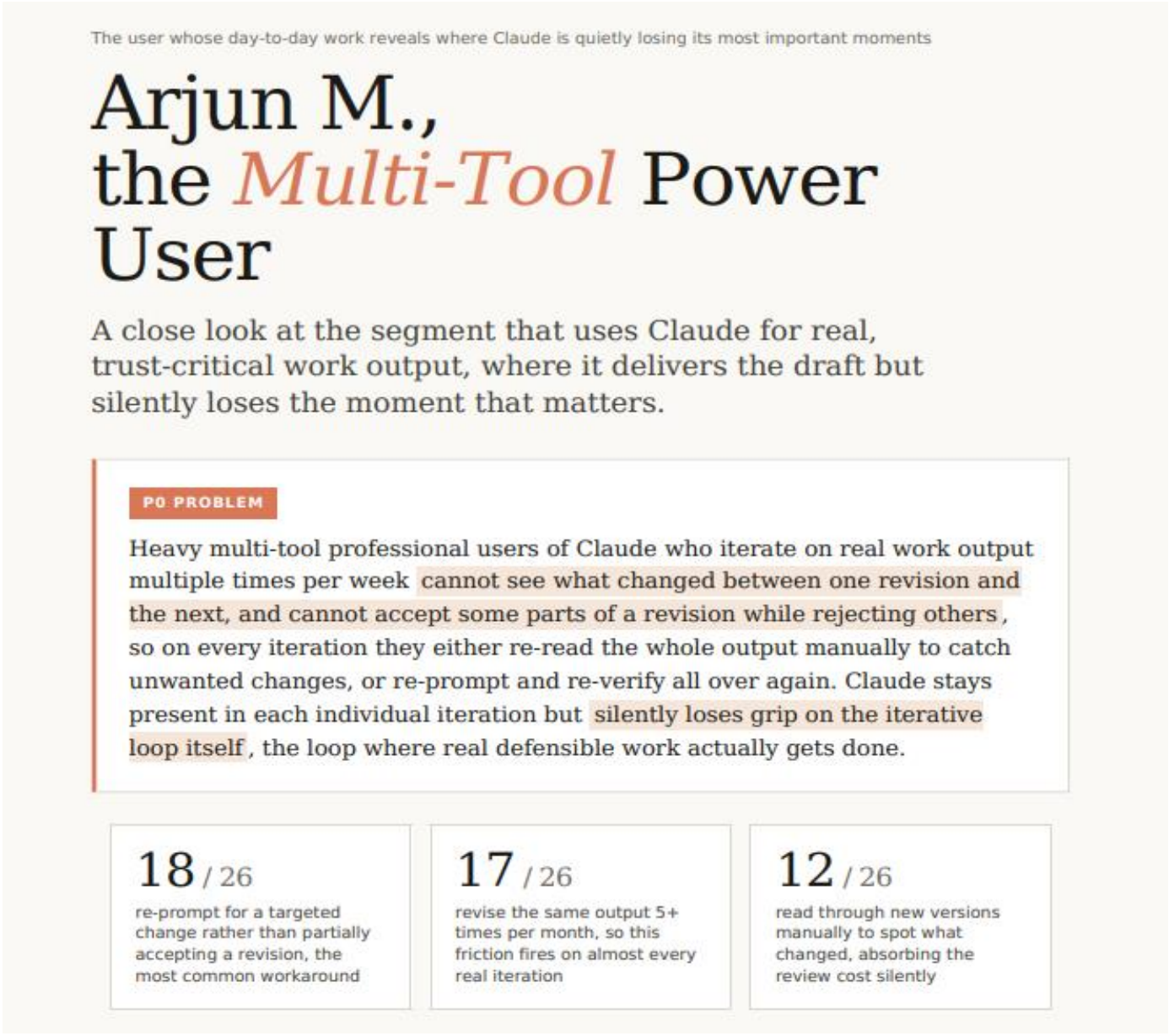
Scope : Phase 3 solution: Version Difference + Section Level Accept/Reject, inside Artifacts

1. Summary

We are fixing the moment a Claude user stops trusting their own revision loop. Today, heavy multi tool professional users who iterate on the same output multiple times a week cannot see what changed between one version and the next, and cannot keep part of a revision while rejecting the rest. So, on every round they either re-read the whole document manually or re prompt Claude and re verify from scratch, quietly absorbing that cost every single time.

We fix this with one integrated capability inside Artifacts: every revision renders as a section level diff, each changed section carries its own keep new or keep old control, and a version history panel makes any prior version inspectable and one click away from being restored. Claude already treats Artifacts as persistent, editable documents; what's missing is that persistence never became memory. This closes that gap directly, so the review cost that is happening today, silently, on every iteration, gets absorbed by the product instead of by the user.

2. Problem and Evidence



The strongest numbers behind it

- 18 of 26 survey respondents re-prompt Claude for just the specific change they wanted rather than partially accepting a revision, the single most common workaround in the entire study.
- 12 of 26 track what changed between versions by reading the new one manually against memory of the last, with no product support at all.
- 17 of 26 revise the same output 5 or more times a month, so this friction fires on almost every real iteration, not an occasional edge case.
- High-frequency revisers rate this pain lower (2.24) than low-frequency users (2.67) on a direct severity scale, the classic signature of a workaround so routine it has stopped registering as a problem.
- A discovered variant (Insight 7): Claude sometimes over-generalizes a targeted edit into a sweeping one, deepening the gap from "can't see what changed" into "can't control the scope of what changes."

3. User and Context

User Persona

A

Arjun M.

THE MULTI-TOOL POWER USER · 32 · ENGINEER-ANALYST HYBRID

Uses Claude, ChatGPT, and one other tool (Gemini or Perplexity) daily across coding, research, drafts, and analysis. Access is a mix of paid Pro and employer plans. He is not a casual user, and not a solo Claude Code developer either. He is the user whose day-to-day work sits at the intersection of every pain point Phase 2 surfaced.

> *"When I ask Claude to change one part, I never know what else quietly moved. So I re-read the whole thing every time. Or I just re-prompt and hope.*

Goals & motivations

- Get real work done faster than solo, but with output he can personally stand behind
- Route work to the right tool for the job: Claude for reasoning, others for search or content
- Stay within plan limits without either constantly worrying about them or under-using what he pays for

Actual behaviors

- Iterates on the same output 5+ times per month **17 of 26**
- Re-prompts for the specific change he wanted rather than accepting the whole revision **18 of 26**
- Reads new versions manually against memory to spot unwanted changes **12 of 26**
- Cross-checks reasoning with a second AI on trust-critical output **14 of 26**
- Runs separate AI conversations when a colleague is also involved **5 of 26**

FRUSTRATIONS, IN THEIR OWN WORDS

SURVEY 18 / 26	*Asked the AI to redo it with just the changes I wanted.*
SURVEY 12 / 26	*Read through the new version alone and spotted differences from own memory.*
OPEN TEXT + Priyank	*AI on its own updated everything that was similar to the request despite the user not mentioning about other similar stuff.*

User Journey

Arjun works on a client-facing deliverable in Claude. Seven stages, from first draft to sending it out. The tracking friction fires every iteration, silently, and the review cost accumulates across the loop until Arjun starts doing the higher-value parts himself.

Works well · Silent friction · absorbed by user, invisible to Claude · Highest-value drop-off · the moment that matters

1 Draft
Uploads a brief. Asks Claude for a first draft.

WORKS WELL
The first output impresses. This is where Claude shines.

2 Ask for a targeted revision
"Improve the executive summary and tighten the recommendations section."

SILENT FRICTION
Full document comes back regenerated. No diff shown against the previous version.

3 Verify what changed
Reads the new draft against memory of the last one, trying to spot which paragraphs actually moved.

SILENT FRICTION · 12 / 26
Cannot tell whether small changes elsewhere are intentional refinements or unrequested rewrites. Absorbs the review cost every round.

4 Partial acceptance, blocked
Wants to keep the new executive summary but reject the tightened recommendations. No mechanism exists to accept some parts and reject others.

P0 · HIGHEST-EVIDENCE FRICTION · 18 / 26
Re-prompts asking for just what he wanted, the most common workaround in the whole dataset. Now has to re-verify the whole document again. Loop repeats every revision cycle.

5 Cascading edit surprise
Notices Claude has also "tightened" a paragraph he never asked to touch, in the style of the recommendations revision, because it looked similar.

P0 SHARPENER · scope-control variant
The user asked to change one part; Claude generalised the change to similar-looking parts unprompted. Deepens the tracking gap into a control gap.

6 Fact-check the output
The final version cites correct-looking sources; Arjun spot-checks the reasoning before sending.

SECONDARY FRICTION · 14 / 26
14 of 26 rejected an AI answer for reasoning that didn't add up. Notable but downstream of the track-changes loop, and Priyank noted this has improved with recent Claude updates.

7 Send to client
Depending on how many silent changes and cascading edits accumulated across revisions, either sends Claude's version or manually rewrites the parts he no longer trusts.

OUTCOME
Review cost of stages 3-5 accumulates across every iteration. On a heavily iterated document, Arjun does the higher-value parts himself, not because Claude got them wrong once, but because he lost confidence in tracking what Claude was doing.

THE PATTERN THIS REVEALS

The tracking friction fires every iteration, silently. Stage 4 is where partial acceptance is blocked and the workaround loop begins. Stage 5 deepens it into a scope-control problem. Across 5+ revisions per month, the review cost accumulates until Arjun does the higher-value parts himself, not because Claude got them wrong, but because he lost confidence in tracking what Claude was doing.

5. Solution

The solution is one integrated capability inside Artifacts, not three separate features bolted together. Claude already treats an Artifact as a persistent, editable object; today an update simply overwrites the previous version with no memory of it. This gives that persistence an actual memory.

Version Diff View

- A user is iterating on an Artifact and asks for a revision, for example "tighten the recommendation and soften the intro."
- Instead of silently overwriting the document, Claude returns the revision with a Compare toggle visible at the top of the Artifact panel, active by default on any turn that changed the document.
- Toggling Compare shows the document with only the changed sections expanded to a diff view: removed text struck through, added text highlighted, unchanged sections collapsed to their normal reading state.
- The user sees exactly which sections changed and which didn't, without needing to hold the previous version in memory.

Section-Level Accept / Reject

- Each changed section carries two controls directly beneath it: Keep new and Keep old.
- Picking Keep old on a section immediately reverts just that section to its prior wording and opens a small inline box: "tell Claude what to change here instead," so a rejected section can be re-specified without reopening the whole document or losing the sections that were already accepted.
- Accept all / Reject all controls sit in the compare banner for the common case where every change is either good or not.
- Once every changed section has a decision, Confirm changes finalizes the document as a new version.

Version History Panel

- A History control opens a simple, chronological list of every past version of the Artifact.
- Each entry can be previewed read-only, or restored directly as the current version in one click, no re-prompting Claude to reconstruct something it already produced once.
- This is the container the diff view and section controls live inside: version N is always diffable against version N-1, and any version is always one click from being current again.

Prototype: version-diff-prototype.html (attached alongside this document)

Walkthrough video: 2 to 5 minutes, to be recorded separately, covering the P0 in one line, then clicking through the flow above end to end, from requesting a revision through confirming a mixed accept/reject decision and reverting to a prior version.

6. Requirements

Story 1. Diff view on revision

As a user revising an Artifact, I want to see exactly what changed against the prior version, so that I don't have to re-read the whole document to catch unwanted changes.

- The Compare toggle appears automatically on any Artifact turn that changed existing content, and is on by default.
- Changed sections show removed text struck through and added text highlighted, within the section boundary Claude actually changed.
- Unchanged sections render normally, with no diff markup, so the reading experience stays clean where nothing moved.

Story 2. Section-level accept / reject

As a user reviewing a revision, I want to keep some changed sections and reject others independently, so that I don't lose good changes while fixing bad ones.

- Every changed section has its own Keep new / Keep old control, decidable independently of every other changed section.
- Rejecting a section reverts only that section; every other section keeps its own independent state.
- Confirm changes is disabled until every changed section has an explicit decision, so a mixed, half-decided state is never silently finalized.

Story 3. Rescoping a rejected section

As a user who rejected a section, I want to tell Claude specifically what I want instead, so I don't have to reopen the whole document and repeat context Claude already had.

- Rejecting a section surfaces an inline instruction box scoped to that section only.
- An instruction submitted here only affects the one section it's attached to; every other already-decided section is untouched by the follow-up generation.

Story 4. Version history and revert

As a user who wants to go back, I want to see every past version and restore one directly, so "undo" doesn't require reconstructing a prompt from memory.

- The History panel lists every saved version in order, most recent first, with a short one-line description of what changed from the version before it.
- Any version can be opened in a read-only preview without affecting the current version.
- Restoring a version makes it the new current version and adds a new history entry recording the revert, rather than deleting anything from the history.

Story 5. Accept-all / reject-all shortcut

As a user who agrees with an entire revision, I want to accept everything in one action, so I'm not forced through section-by-section review when I don't need it.

- Accept all and Reject all are available whenever more than one section changed in a turn.
- Either shortcut still leaves each section individually visible and re-decidable afterward before the user confirms, so a fast decision is never irreversible before confirmation.

Edge cases and empty states

- First version of a new Artifact. No Compare toggle appears and no history entries beyond "Version 1, created" exist, since there is nothing yet to diff against.

- A revision that changes nothing detectable as a distinct section (for example, a single word fixed mid-paragraph). The whole paragraph is treated as the section boundary rather than leaving an un-reviewable fragment; sub-paragraph diffing is not attempted in this release.
- A revision that touches every section at once. All sections show as changed and the Accept all / Reject all shortcuts become the primary path, since section-by-section review of a full rewrite provides little benefit over reviewing the whole document directly.
- User closes the Artifact mid-review, with some sections decided and others not. The undecided state is preserved and resumed exactly as left when the Artifact is reopened; nothing is auto-confirmed by navigating away.
- User rejects a section, then rejects the rescoped replacement Claude generates for it. The section can be rejected any number of times in sequence; each rejection reopens the inline instruction box rather than escalating to any special state.
- Reverting to a much older version when many versions exist. The revert always operates on the full document state at that version, not a merge with anything in between, and is itself recorded as a new version rather than deleting the intervening history.
- Very long documents with many changed sections. The compare banner's Accept all / Reject all remains available regardless of section count, so review effort doesn't scale linearly with document length for users who trust the whole revision.

7. Success Metrics

Kind	What is measured	Why this one
Key	Share of Artifact revisions where the user makes section-level accept/reject decisions through the diff mechanism, rather than re-prompting Claude to redo the whole thing.	This is the exact behaviour the P0 describes. If re-prompting stays just as common as it is today, the feature has not actually closed the gap, no matter how often it gets opened.
Secondary	Diff-view engagement rate (share of revisions where Compare is actually used) and manual re-read rate immediately following a revision, expected to fall as the diff absorbs that need.	Engagement is the funnel step before the key metric can move at all. The re-read rate is the other current workaround (12 of 26); if it doesn't fall even as engagement rises, the diff isn't yet trusted as a substitute for manually checking.
Guardrail	Time-to-next-action after a revision, and the share of revision sessions that end in a finished, exported Artifact rather than one abandoned mid-review.	A review step that adds friction to the exact loop it's meant to speed up would be a net loss even if engagement looks healthy. If either of these worsens materially, the mechanism is slowing iteration down rather than restoring trust in it.

8. Risks, Dependencies, Assumptions and Open Questions

Assumptions

- Changes are naturally separable into discrete, independently judgeable sections, rather than diffuse edits scattered across a document with no clean boundary.
- A visual diff is the format users actually want when asking "what changed," not a written summary of the same information.
- Section-level accept/reject genuinely reduces re-prompting, rather than simply relocating the same manual effort into a new interface.
- Users will engage with a version history panel at all, rather than only ever caring about the immediately previous version.

Dependencies

- Claude's generation behaviour needs to reliably preserve untouched text verbatim on a targeted revision, so the diff has real section boundaries to render rather than one large, unreadable changed block.
- Artifacts' underlying storage needs to retain every version, not just the current state, which is a real infrastructure change from today's overwrite-in-place model.
- Design for how a diff renders inside code Artifacts specifically may differ meaningfully from prose Artifacts, and needs its own pass; this PRD scopes to prose/document Artifacts first.

Risks

- The diff could render as one large, unreadable changed block if Claude's rewrite behaviour doesn't respect natural section boundaries, collapsing section-level review back into effectively all-or-nothing. Medium likelihood, high severity. Early signal: fewer than 2 distinct change regions on more than half of revisions in early testing. No-engineering response: tune the revision prompt to explicitly preserve untouched text verbatim, tested manually on a sample of real revision requests before any UI ships.
- Users might find the visual diff noisy and revert to manually reading the whole document anyway, in which case diff-view engagement looks healthy while the manual re-read guardrail never actually falls. Medium likelihood, high severity. Response: offer an optional plain-language change summary alongside the visual diff, and let real usage decide which format actually reduces manual re-reading.
- Section-level accept/reject might not reduce re-prompting at all if rejecting a section doesn't give Claude enough to act on, so the user still types a follow-up prompt anyway, just inside a new interface. Medium likelihood, medium severity. Response: the inline rescoping box (Story 3) exists specifically to keep that follow-up instruction scoped to the one rejected section rather than reopening the whole document.
- Version history could go largely unused if the section-level workflow alone already covers what most users need, making the history panel a low-value surface to have built. Low likelihood, low severity. No-engineering response: ship it, but do not invest further design effort expanding it until usage data justifies doing so; low usage here is an acceptable, cheap outcome to learn from.

Open questions

- Should the Compare view default to on for every revision, or only when Claude judges the change was non-trivial? Defaulting on for everything may create review fatigue on small, obviously-fine edits. A Design and Research decision, informed by the diff-engagement metric once real usage exists.
- How many past versions should be retained indefinitely versus rolled into a summarized older-versions state, and at what point does storage cost start to matter? An Engineering and Infrastructure decision.
- Should scoped edit mode (Idea 3, pre selecting what Claude may touch before generating) ship as a natural Phase 4 extension of this same version model, or does it need to be evaluated independently against fresh usage data first? A Product decision to revisit once this release has real adoption numbers.
- Does the cascading-edit warning (Idea 6) become worth pursuing once real telemetry shows how often over-broad edits actually occur in practice, now that the diff view provides an after-the-fact detection mechanism? Needs a dedicated Research pass once this release has shipped and generated real data on the problem's true frequency.